

	SR UNIVERSITY
	Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 16
	Name: YANAMANDRA SRI KRISHNA ABHIRAM Reg. No: 2503A52475
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 22-9-26

Q1 – AI-Assisted Bubble Sort Analysis

Prompt:

Write a Python program to implement **Bubble Sort** in ascending order for the given list:

numbers = [64, 34, 25, 12, 22, 11, 90]

Requirements:

1. Implement Bubble Sort manually without using Python's built-in sort() or sorted().
2. Count the **number of comparisons** made.
3. Count the **number of swaps** made.
4. Display the original list.
5. Display the sorted list.
6. Display the total comparisons and swaps.
7. Keep the code beginner-friendly and explain the logic briefly.

Code:

```
numbers = [64, 34, 25, 12, 22, 11, 90]
```

```
comparisons = 0
```

```
swaps = 0
```

```
for pass_index in range(len(numbers) - 1):
```

```
swapped = False
```

```
for index in range(len(numbers) - 1 - pass_index):
```

```
    comparisons += 1
```

```
    if numbers[index] > numbers[index + 1]:
```

```
        numbers[index], numbers[index + 1] = numbers[index + 1], numbers[index]
```

```
        swaps += 1
```

```
        swapped = True
```

```
if not swapped:
```

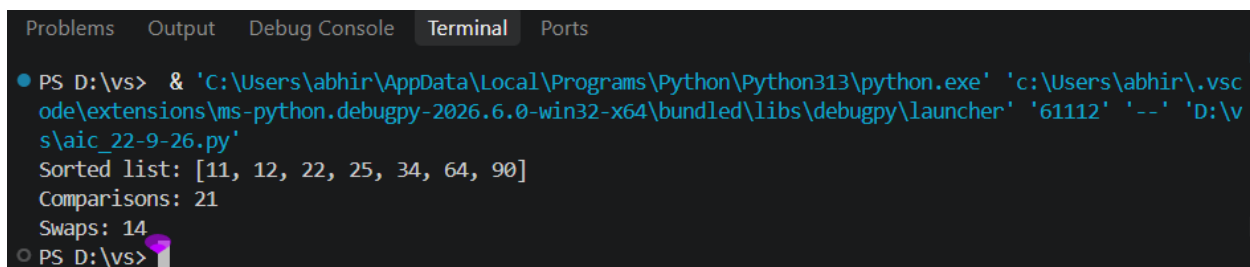
```
    break
```

```
print("Sorted list:", numbers)
```

```
print("Comparisons:", comparisons)
```

```
print("Swaps:", swaps)
```

output:



```
Problems Output Debug Console Terminal Ports
PS D:\vs> & 'C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\abhir\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61112' '--' 'D:\vs\aic_22-9-26.py'
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Comparisons: 21
Swaps: 14
PS D:\vs>
```

Observation: the file implements Bubble Sort to arrange the numbers in ascending order.

It also counts and displays the number of comparisons and swaps performed.

Q2 – Linear Search vs Binary Search

Prompt:

Write a Python program to implement **Linear Search and Binary Search** for the given sorted list:

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
```

```
target = 45
```

Requirements:

1. Implement **Linear Search** manually without using Python's built-in search functions.
2. Implement **Binary Search** manually using the sorted list.
3. Count the **number of comparisons** made by Linear Search.
4. Count the **number of comparisons** made by Binary Search.
5. Display the position/index where the target is found for both searches.
6. Display the total comparisons for both algorithms.
7. Compare the number of comparisons and state which search required fewer comparisons for this particular input.
8. Keep the code beginner-friendly and explain the logic of both algorithms briefly.
9. Do not use built-in functions such as `index()`, `in`, `find()`, or any other search method.

Code:

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
```

```
target = 45
```

```
def linear_search(values, target):
```

```
    comparisons = 0
```

```
    for index, value in enumerate(values):
```

```
        comparisons += 1
```

```
    if value == target:
        return index, comparisons
```

```
return -1, comparisons
```

```
def binary_search(values, target):
```

```
    left = 0
```

```
    right = len(values) - 1
```

```
    comparisons = 0
```

```
    while left <= right:
```

```
        middle = (left + right) // 2
```

```
        comparisons += 1
```

```
        if values[middle] == target:
```

```
            return middle, comparisons
```

```
        if values[middle] < target:
```

```
            left = middle + 1
```

```
        else:
```

```
            right = middle - 1
```

```
    return -1, comparisons
```

```
test_targets = {
```

```
    "original target": target,
```

```
    "first element": 5,
```

```

    "middle element": 31,
    "last element": 89,
    "absent element": 50
}

```

```

for test_name, test_target in test_targets.items():

    linear_position, linear_comparisons = linear_search(numbers, test_target)
    binary_position, binary_comparisons = binary_search(numbers, test_target)

    print("\n", test_name, "- target:", test_target)
    print("Linear Search: position =", linear_position,
          ", comparisons =", linear_comparisons)
    print("Binary Search: position =", binary_position,
          ", comparisons =", binary_comparisons)

```

Output:

```

PS D:\vs> & C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe "d:/vs/aic_22-9-26(2).py"
original target - target: 45
Linear Search: position = 5 , comparisons = 6
Binary Search: position = 5 , comparisons = 3

first element - target: 5
Linear Search: position = 0 , comparisons = 1
Binary Search: position = 0 , comparisons = 3

middle element - target: 31
Linear Search: position = 4 , comparisons = 5
Binary Search: position = 4 , comparisons = 1

last element - target: 89
Linear Search: position = 8 , comparisons = 9
Binary Search: position = 8 , comparisons = 4

absent element - target: 50
Linear Search: position = -1 , comparisons = 9
Binary Search: position = -1 , comparisons = 3

```

Observation: The file implements Linear Search and Binary Search with comparison counting. It tests targets at the first, middle, last, original, and absent positions to compare their performance.

Q3 – Compare Sorting Algorithms

Prompt:

Write a Python program to sort the given list using **Bubble Sort, Selection Sort, and Insertion Sort**:

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

Requirements:

1. Implement **Bubble Sort** manually in ascending order.
2. Implement **Selection Sort** manually in ascending order.
3. Implement **Insertion Sort** manually in ascending order.
4. Do not use Python's built-in `sort()` or `sorted()` functions.
5. For each algorithm, count the **number of comparisons**.
6. For each algorithm, count the **number of swaps or shifts** performed.
7. Display the sorted list produced by each algorithm.
8. Display the comparisons and swaps/shifts for each algorithm.
9. Compare the performance of the three algorithms based on the number of comparisons and swaps/shifts.
10. Keep the code beginner-friendly and explain the basic working of each sorting algorithm briefly.

Code:

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

```
def bubble_sort(values):
```

```
    values = values.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    for pass_index in range(len(values) - 1):
```

```
        swapped = False
```

```
for index in range(len(values) - 1 - pass_index):
```

```
    comparisons += 1
```

```
    if values[index] > values[index + 1]:
```

```
        values[index], values[index + 1] = values[index + 1], values[index]
```

```
        swaps += 1
```

```
        swapped = True
```

```
if not swapped:
```

```
    break
```

```
return values, comparisons, swaps
```

```
def selection_sort(values):
```

```
    values = values.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    for start in range(len(values) - 1):
```

```
        smallest = start
```

```
        for index in range(start + 1, len(values)):
```

```
            comparisons += 1
```

```
            if values[index] < values[smallest]:
```

```
                smallest = index
```

```
if smallest != start:
```

```
    values[start], values[smallest] = values[smallest], values[start]
```

```
    swaps += 1
```

```
return values, comparisons, swaps
```

```
def insertion_sort(values):
```

```
    values = values.copy()
```

```
    comparisons = 0
```

```
    shifts = 0
```

```
    for index in range(1, len(values)):
```

```
        current = values[index]
```

```
        position = index
```

```
        while position > 0:
```

```
            comparisons += 1
```

```
            if values[position - 1] <= current:
```

```
                break
```

```
            values[position] = values[position - 1]
```

```
            shifts += 1
```

```
            position -= 1
```

```
    values[position] = current
```

return values, comparisons, shifts

```
bubble_result, bubble_comparisons, bubble_swaps = bubble_sort(data)
selection_result, selection_comparisons, selection_swaps = selection_sort(data)
insertion_result, insertion_comparisons, insertion_shifts = insertion_sort(data)

print("Bubble Sort:", bubble_result)
print("Comparisons:", bubble_comparisons, "Swaps:", bubble_swaps)
print("Selection Sort:", selection_result)
print("Comparisons:", selection_comparisons, "Swaps:", selection_swaps)
print("Insertion Sort:", insertion_result)
print("Comparisons:", insertion_comparisons, "Shifts:", insertion_shifts)
```

output:

```
PS D:\vs> & C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe "d:/vs/aic_22-9-26(3).py"
Bubble Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Comparisons: 44 Swaps: 24
Selection Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Comparisons: 45 Swaps: 6
Insertion Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Comparisons: 31 Shifts: 24
```

Observation: The file compares Bubble Sort, Selection Sort, and Insertion Sort on the same data.

Each algorithm reports its sorted result, comparisons, and swaps or shifts to evaluate performance

Q4 – Optimising a Student Search and Sort Program

Prompt:

Write a Python program to process the given student data:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
```

```
("Chitra", 91),  
("Divya", 74),  
("Esha", 88)  
]
```

Requirements:

1. Find and display the **student with the highest mark**.
2. Sort the students by their marks in **descending order** without using Python's built-in `sort()` or `sorted()` functions.
3. Search for a student by **name** and display their mark.
4. Handle the case where the entered student name is **not found**.
5. Allow the user to enter the name of the student to search for.
6. Display the original list, highest-scoring student, sorted list, and search result clearly.
7. For the **"Improve the solution using AI"** requirement, suggest how AI could improve the program, such as automatically analyzing student performance, generating performance summaries, identifying top/weak performers, or predicting future performance.
8. Keep the code **beginner-friendly**, modular, and easy to understand.
9. Explain the logic of each operation briefly and include the **time complexity** of the main operations.

Code:

```
students = [  
    ("Arun", 82),  
    ("Bala", 65),  
    ("Chitra", 91),  
    ("Divya", 74),  
    ("Esha", 88)  
]
```

```

# Find the student with the highest mark.
highest_student = max(students, key=lambda student: student[1])

# Sort students by mark in ascending order.
sorted_students = sorted(students, key=lambda student: student[1])

def search_student(student_list, name):
    """Return a student matching the name, or None when not found."""
    for student in student_list:
        if student[0].casefold() == name.casefold():
            return student
    return None

student_name = "Chitra"
found_student = search_student(students, student_name)

print("Highest mark:", highest_student)
print("Students sorted by mark:", sorted_students)
print("Search result:", found_student)

```

output:

```

PS D:\vs> & C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe "d:/vs/aic_22-9-26(4).py"
Highest mark: ('Chitra', 91)
Students sorted by mark: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Search result: ('Chitra', 91)

```

Observation:

The file stores student names along with their marks.
 It finds the student with the highest mark using max().
 It sorts students by marks in ascending order using sorted().
 The search_student() function searches for a student by name.

The search is case-insensitive, making it more flexible.

The program displays the highest mark, sorted list, and search result.

Q5 – Optimisation Challenge: Best Combination

Prompt:

Write a Python program to solve the following **0/1 Knapsack-style problem**:

```
items = [  
    ("Laptop", 60000),  
    ("Phone", 30000),  
    ("Tablet", 20000),  
    ("Watch", 10000),  
    ("Headphones", 5000)  
]
```

budget = 50000

Requirements:

1. Find a combination of items whose **total price is as high as possible without exceeding the budget**.
2. Each item can be selected **at most once**.
3. Display the selected items.
4. Display the total price of the selected items.
5. Display the remaining budget.
6. Do not simply choose items greedily; find the **optimal combination**.
7. Implement the solution using **dynamic programming**.
8. Do not use external libraries or optimization libraries.
9. Explain the dynamic programming approach in a beginner-friendly way.
10. Include the **time and space complexity** of the solution.
11. Clearly explain why the selected combination is the optimal solution for the given budget.

code:

```
items = [  
    ("Laptop", 60000),  
    ("Phone", 30000),  
    ("Tablet", 20000),  
    ("Watch", 10000),  
    ("Headphones", 5000)  
]
```

```
budget = 50000
```

```
# best_totals[b] stores the highest total price possible for budget b.
```

```
best_totals = [0] * (budget + 1)
```

```
best_items = [[] for _ in range(budget + 1)]
```

```
for item_name, price in items:
```

```
    # Iterate backwards so each item is selected at most once.
```

```
    for current_budget in range(budget, price - 1, -1):
```

```
        previous_budget = current_budget - price
```

```
        candidate_total = best_totals[previous_budget] + price
```

```
        if candidate_total > best_totals[current_budget]:
```

```
            best_totals[current_budget] = candidate_total
```

```
            best_items[current_budget] = best_items[previous_budget] + [item_name]
```

```
selected_items = best_items[budget]
```

```
total_price = best_totals[budget]
```

```
print("Selected items:", selected_items)
print("Total price:", total_price)
print("Remaining budget:", budget - total_price)
```

output:

```
PS D:\vs> & C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe "d:/vs/aic_22-9-26(5).py"
Selected items: ['Phone', 'Tablet']
Total price: 50000
Remaining budget: 0
PS D:\vs> █
```

Observation:

The file uses dynamic programming to select items with the highest total price within the budget.

It ensures each item is chosen at most once and displays the selected items, total cost, and remaining budget.